



MASTER DI I LIVELLO **CYBERSECURITY**

UNIVERSITÀ DEGLI STUDI DI TORINO



Cod. Corso A757- 12-2024-0
ID Attività 2584660

Insegnamento: Smart Contract Security in Ethereum

Prof. Silvio Meneguzzo

MODULE

Smart Contract Security in Ethereum

Best practices, antipatterns and real-world exploits: reentrancy, DELEGATECALL, randomness, front-running, DoS, precision, oracle/price manipulation, access control, signature replay & misconfiguration

Blockchain and Distributed Ledger Technologies

Structured on Ch. 9 of “Mastering Ethereum” (2nd ed.), Antonopoulos, Wood et al. — CC BY-NC-ND

Outline

- | | | |
|-----|---|--|
| I | Foundations & best practices | Why contract security is special; defense-in-depth; defensive programming |
| II | Antipatterns & exploits | Reentrancy, DELEGATECALL, randomness, unchecked calls, front-running, DoS, ... |
| III | More vulnerability classes | Precision, price/oracle manipulation, input validation, signatures, misconfig |
| IV | Tooling & resources | OpenZeppelin / Solmate / Solady; security wargames; key takeaways |
| V | References | Mastering Ethereum, EIPs, OpenZeppelin, exploit post-mortems |

Are blockchains secure?

YES, but...

The total amount of money lost by
blockchain hackers is about
\$ 26,648,918,145.89
Total hack events 783

<https://hacked.slowmist.io/en/>

Blockchain security concerns

Smart
Contracts

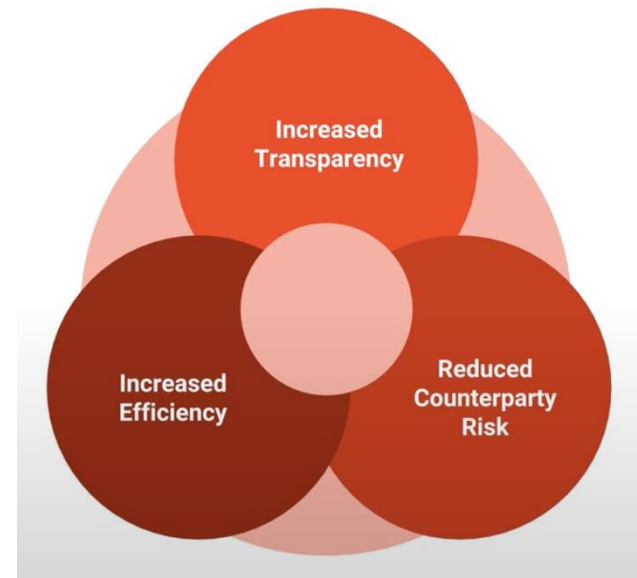
Infrastruc
ture
Protocol

Off-chain
code

Humans

Smart Contract Security

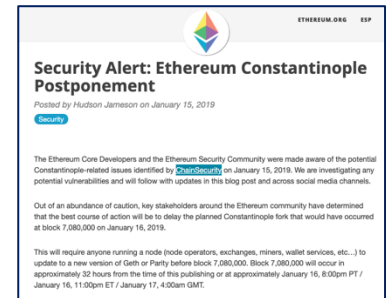
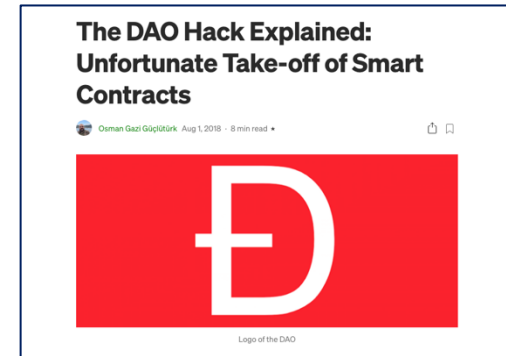
- Public accessible
- Can hold value
- Immutable -> cannot fix bugs
 - Some patterns based on proxies has been proposed
 - OpenZeppelin



<https://docs.openzeppelin.com/upgrade-plugins/1.x/writing-upgradeable>

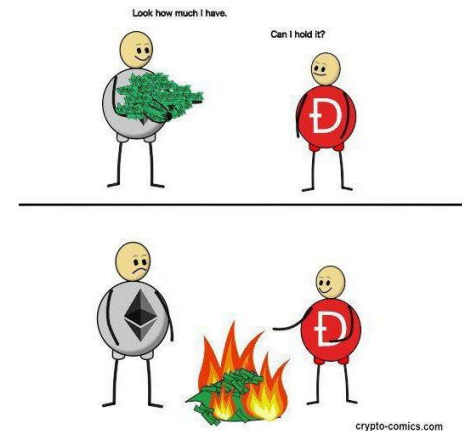
Smart Contracts Security Incidents

- 2016: The DAO hack
 - \$ 70 M lost
- 2017: Parity Multisig
 - \$ 300 M frozen
- 2019: Ethereum Constantinople Update Postponed
- 2020: Ethereum Istanbul Update
 - Some deployed contracts stop working



The DAO hack

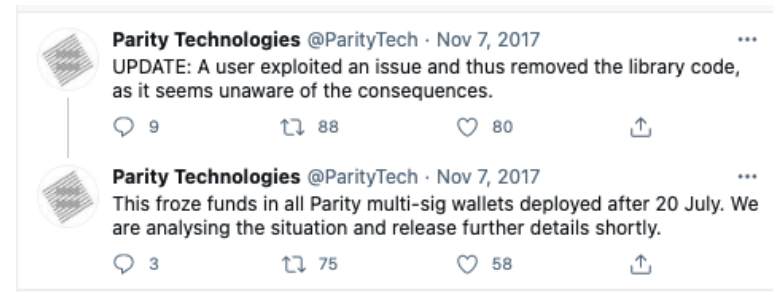
- The DAO
 - Crowdsourced fund
 - 12.7M Ether (\$150M) in a single smart contract address
 - Almost 15% existing Ether at the time
- The attack
 - June 17, 2016
 - **Reentrancy vulnerability** in the contract code
 - Child DAO creation with 28-day lock
 - 28 days to recover funds
- Fallout
 - Hard fork in Ethereum blockchain
 - Temporary Ether price crash
 - The DAO collapse



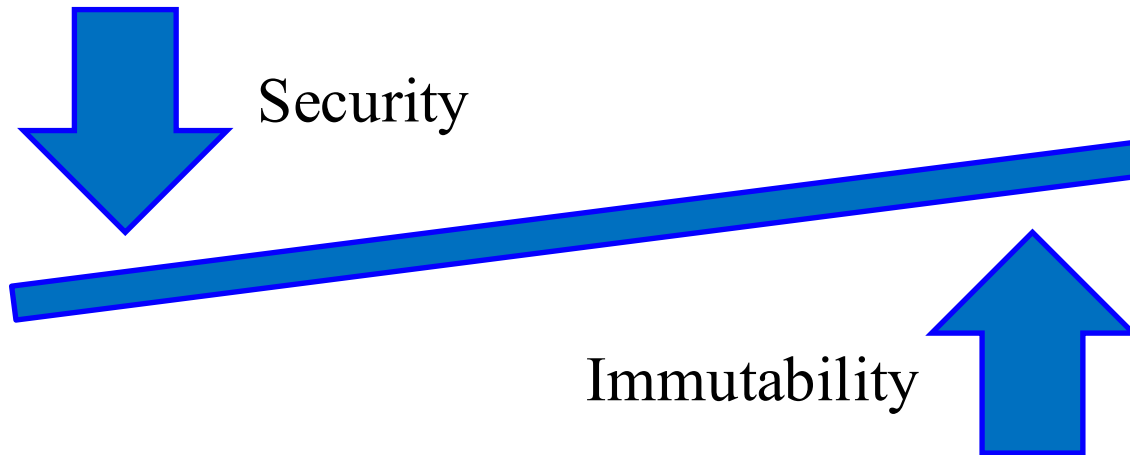
Parity Multisig Wallet hack



- Parity was an online wallet service since Ethereum's start
- Multiple types of wallets, including Multi Signature (MultiSig) wallets
- The Smart Contract handling Parity MultiSig wallets promised transparency of operations, autonomy (lack of Parity's control over user funds), and speed.
- As part of its lengthy design, it was written to rely on some libraries, that in Ethereum are Smart Contract too
- One of the libraries the Parity Smart Contract relied on was left owner-less by mistake.
- A user was able to hijack it harmlessly, and become the owner.
- He then told the library to commit suicide (possibly by mistake) and broke the Parity contract that relied on this library.
- **Result: 513,743 ETH (\$280M) of user funds remain permanently “frozen”, until today.**



Protocol Update Dilemma



Solution: Smart Contract Auditing



- The community is maintaining different well known databases of smart contract vulnerabilities
 - Example: Consensys (<https://consensys.github.io/smart-contract-best-practices/attacks/>)
- Provided by **experts**
 - Expensive
 - Usually done at the end of the development
- Provided by **Automatic analysys**
 - Dataflow analysis
 - Symbolic execution
 - Deductive verification
 - ...

New batchOverflow Bug in Multiple ERC20 Smart Contracts (CVE-2018-10299)

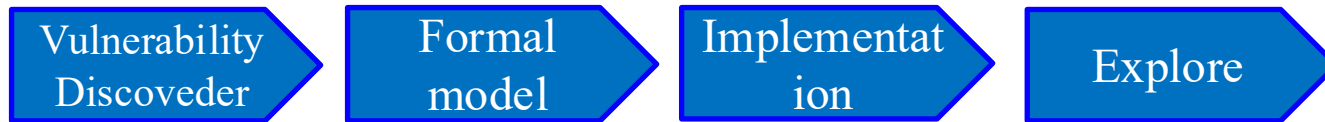


Built on our earlier efforts in analyzing EOS tokens, we have developed an automated system to scan and analyze Ethereum-based (ERC-20) token transfers. Specifically, our system will automatically send out alerts if any suspicious transactions (e.g., involving unreasonably large tokens) occur.

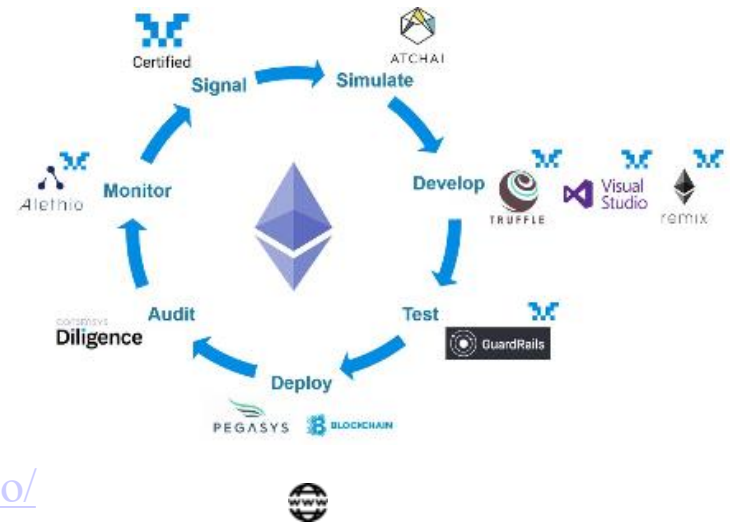
SWC Registry			
Smart Contract Weakness Classification and Test Cases			
The following table contains an overview of the SWC registry. Each row consists of an SWC identifier (ID), weakness title, CWE parent and list of related code samples. The links in the ID and Test Cases columns link to the respective SWC definition. Links in the Relationships column link to the CWE Base or Class type.			
ID	Title	Relationships	Test cases
SWC-136	Unencrypted Private Data On-Chain	CWE-767: Access to Critical Private Variable via Public Method	• odd_even.sol • odd_even_fixed.sol
SWC-135	Code With No Effects	CWE-1164: Irrelevant Code	• deposit_box.sol • deposit_box_fixed.sol • wallet.sol • wallet_fixed.sol
SWC-134	Message call with hardcoded gas amount	CWE-655: Improper Initialization	• hardcoded_gas_limits.sol
SWC-133	Hash Collisions With Multiple Variable Length Arguments	CWE-294: Authentication Bypass by Capture-replay	• access_control.sol • access_control_fixed_1.sol • access_control_fixed_2.sol
SWC-132	Unexpected Ether balance	CWE-667: Improper Locking	• Lockdrop.sol
SWC-131	Presence of unused variables	CWE-1164: Irrelevant Code	• unused_state_variables.sol • unused_state_variables_fixed.sol • unused_variables.sol • unused_variables_fixed.sol
SWC-130	Right-To-Left-Override control character (U+202E)	CWE-451: User Interface (UI) Misrepresentation of Critical Information	• guess_the_number.sol
SWC-129	Typographical Error	CWE-480: Use of Incorrect Operator	• typo_one_command.sol • typo_safe_math.sol • typo_simple.sol
SWC-128	Dos With Block Gas Limit	CWE-400: Uncontrolled Resource Consumption	• dos_address.sol • dos_number.sol • dos_simple.sol
SWC-127	Arbitrary Jump with Function Type Variable	CWE-695: Use of Low-Level Functionality	• FunctionTypes.sol

<https://swcregistry.io/>

Solution: Auditing with automatic analysis



- Security can be checked from the beginning
- Development
 - Quick iteration
 - Thorough code review
- Risk assessment
 - Certification
 - Clarification



<https://mythx.io/>

PART I

Foundations & best practices

Why smart-contract bugs are uniquely dangerous — and how to write defensively

Why smart-contract security is different

A contract executes exactly what is written — not what you intended. Four properties make every mistake unusually costly:

Immutable

Once deployed, code usually cannot be changed. Even upgradeable contracts give you little time to react before an exploit completes.

Public & transparent

Anyone can read the bytecode and call any function. There are no private endpoints; obscurity is not security.

Adversarial

Every input is attacker-controlled. Never assume arguments are well-formed, bounded, or benign.

Money, irreversible

Bugs map directly to lost funds, and losses are almost always unrecoverable. An exploit can finish in a single transaction.

Defense in depth: the “Swiss cheese” model

No single safeguard is perfect — each is a slice of cheese with holes. Stacked together, the holes rarely line up. Security is built in layers; solid development practices are the FIRST and most important slice.

1. Solid development practices

Reliable patterns, clear code, avoiding known pitfalls — the foundation.

2. Thorough testing

Unit, integration, fuzzing and invariant tests against adversarial inputs.

3. Code reviews & audits

Independent expert eyes; public, open-source review.

4. Bug bounties & monitoring

Economic incentive to disclose; on-chain monitoring and pausing.

Defensive programming — best practices

Minimalism / simplicity

Less code → fewer bugs. Question every component; remove edge cases and non-essential features. Simple contracts are easier to reason about and audit.

Code reuse (DRY)

Don't reinvent the wheel. Battle-tested libraries (OpenZeppelin) beat fresh code. Beware “not-invented-here” syndrome.

Code quality

Treat it like aerospace, not a web widget. Every bug can lose money and you cannot easily patch after launch.

Readability / auditability

Clear, well-documented code following community conventions. Develop in the open to harness collective review.

Test coverage

Test everything; validate every argument's range and format. Assume inputs are hostile.

The takeaway

Security starts before the first line of code — in the architecture and the discipline you apply to it.

PART II

Antipatterns & exploits

The most common security risks — vulnerability, example, prevention, real case

A map of the antipatterns we will cover

Real exploits usually chain several of these together. For each we examine: the VULNERABILITY, a concrete EXAMPLE, PREVENTION, and a REAL-WORLD case.

1. Reentrancy

2. DELEGATECALL & storage

3. Entropy illusion (bad randomness)

4. Unchecked CALL returns

5. Race conditions / front-running

6. Denial of Service (DoS)

7. Floating-point & precision

8. Price / oracle manipulation

9. Improper input validation

10. Access control

11. Signature replay

12. Misconfiguration

1 • Reentrancy — the vulnerability

Contracts often make **external calls** (e.g. sending ether). The callee can run code — a fallback/receive function or a token hook — that calls BACK into the original contract **before it has updated its state**. The attacker re-enters while the contract is in an inconsistent state and drains it.

EXAMPLE — EtherStore: a vault meant to allow 1 ether/week withdrawals:

```
function withdrawFunds() public {
  require(block.timestamp >= lastWithdraw[msg.sender] + 1 weeks);
  uint256 amt = balances[msg.sender];
  (bool ok, ) = msg.sender.call{value: amt}(""); // ← external call FIRST
  require(ok, "Transfer failed");
  balances[msg.sender] -= amt; // ← state updated AFTER the call (too late)
  lastWithdraw[msg.sender] = block.timestamp;
}
```

The external call on the highlighted line hands control to the attacker before `balances` is decremented.

1 • Reentrancy — the attack & its flow

```
contract Attack {
  EtherStore store;
  function go() external payable {
    store.depositFunds{value: 1 ether}();
    store.withdrawFunds(); // start
  }
  receive() external payable {
    if (address(store).balance >= 1 ether)
      store.withdrawFunds(); // RE-ENTER
  }
}
```

Result: the whole vault is drained in ONE transaction.

- 1 deposit 1 ETH → balances[attacker] = 1
- 2 call withdrawFunds() → passes the check
- 3 EtherStore sends 1 ETH → triggers receive()
- 4 receive() calls withdrawFunds() AGAIN
- 5 balance still = 1 (not yet decremented) → check passes
- 6 loop until the vault is empty, then state finally updates

0x4E8bEb1d76540D3cD0CC2BE372b0747CA01C3B94

EthStore contract
bugged

0x60030E940a583323E21F3Aa63Cd6F025959a78b6

Attack contract

LAB — DO IT NOW

Reproduce, then fix, a reentrancy drain

Open the folder lab-security/ in Remix (VM). First RUN 01-EtherStore_vulnerable.sol to drain a whole vault in one transaction; then complete 02-SafeEtherStore_start.sol (Checks-Effects-Interactions + a reentrancy lock).

1 • Beyond classic reentrancy

Cross-function

Re-enter a DIFFERENT function that shares the same not-yet-updated state.

Cross-contract

Two integrated protocols sharing state; the attack spans both.

Read-only reentrancy

A view function returns STALE data mid-reentrancy. Contract B trusts A's getter while A's state hasn't caught up — and view functions are rarely guarded.

Token hooks

ERC-721 onERC721Received and ERC-777 tokensReceived hooks are reentrancy vectors too, not just ether transfers.

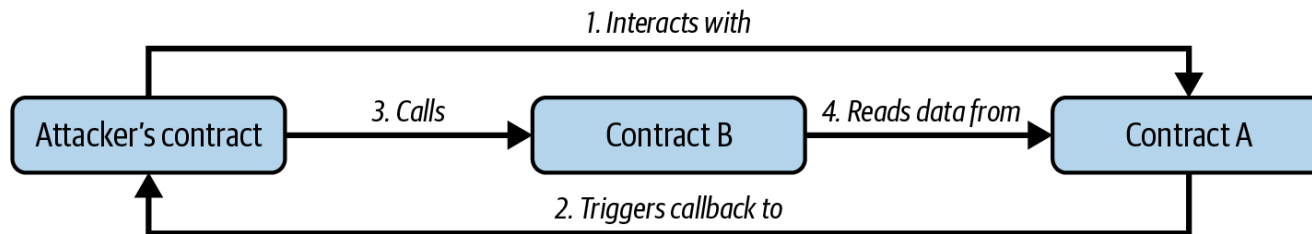


Figure 9-1 — read-only reentrancy: B reads A's stale state during the callback. Source: Mastering Ethereum 2nd ed. (CC BY-NC-ND).

1 • Reentrancy — prevention

Checks-Effects-Interactions — the primary defense: validate (checks), update state (effects), and only THEN call out (interactions). Make external calls the last thing a function does.

```
// fixed order: effects BEFORE the interaction
uint256 amt = balances[msg.sender];
balances[msg.sender] -= amt;           // 1) effect
(bool ok, ) = msg.sender.call{value: amt}(""); // 2) interaction
require(ok, "Transfer failed");
```

Reentrancy lock

A guard modifier (mutex). Prefer OpenZeppelin's ReentrancyGuard — or ReentrancyGuardTransient (EIP-1153 transient storage) for cheaper gas.

transfer / send — with care

They forward only 2,300 gas, blocking most reentry — but can break on contract/EIP-7702 recipients and aren't future-proof. Don't rely on them as the primary defense.

1 • Reentrancy — real-world cases

The DAO (2016)

Held > \$150M (≈15% of all ether). A reentrancy drain led the community to a contentious HARD FORK — splitting the chain into Ethereum and Ethereum Classic (ETC).

Libertify (2023)

A DeFi protocol breached for ~\$400K. A userSwap() path let the attacker re-enter deposit() before totalSupply updated, minting more shares than owed.

Lesson: reentrancy is decades old, easy to spot and cheap to fix — yet still actively exploited. Apply CEI + a guard by default.

2 • CALL vs DELEGATECALL

CALL

Runs the callee's code in the CALLEE's context: its storage, its msg.sender / msg.value. A normal external message call.

DELEGATECALL

Runs the callee's code in the CALLER's context: the caller's storage, and the ORIGINAL msg.sender / msg.value are preserved. The caller “borrows” the callee's bytecode.

DELEGATECALL powers [proxies and libraries](#) (deploy logic once, reuse everywhere). But because it acts on the CALLER's storage, a mismatch between the two contracts' storage layouts is a serious, subtle bug.

Key idea: storage is matched by SLOT NUMBER, not by variable name.

2 • DELEGATECALL — storage-layout hijack

A FibonacciBalance contract delegatecalls a FibonacciLib. Their slot layouts disagree:

```
// Library slots          // Caller slots (what delegatecall actually writes)
slot[0] = start           slot[0] = fibonacciLibrary (an address!)
slot[1] = calculatedFib   slot[1] = calculatedFib
// setStart(x) writes slot[0] → in the caller it overwrites the LIBRARY ADDRESS
```

Exploit: the caller's fallback forwards any call to the library, so an attacker calls `setStart(uint256(attackerContract))`. That overwrites `slot[0]` — the library address — pointing it at the attacker's contract. Every later `withdraw` / `fallback` then delegatecalls ATTACKER code, which runs in the victim's context and can seize all its ether.

2 • DELEGATECALL — prevention & Parity hack

Use the `library` keyword

Solidity libraries are forced stateless and non-self-destructible, removing the storage-context trap.

Mind the calling context

With DELEGATECALL, audit BOTH storage layouts. Prefer stateless logic; use audited proxy patterns (e.g. OpenZeppelin).

Real-world — Parity multisig (2nd hack, 2017). Hundreds of wallets delegated all logic to ONE shared WalletLibrary. That library was itself an uninitialised contract: a user called `initWallet` to become its owner, then called `kill` → `selfdestruct`. Every wallet pointing at it instantly lost all functionality and ether — permanently frozen. The exploiter's GitHub comment: “I accidentally killed it.”

3 · Entropy illusion — bad randomness

Ethereum is **deterministic**: every node must compute the same result, so there is NO native randomness. Using block variables (hash, timestamp, number) as a seed is an antipattern.

Validator manipulation

The block proposer controls/knows block variables. They can reorder or drop transactions to steer an outcome in their favour.

Contract foreknowledge

Other contracts can read the same block variables in the same block and choose to interact only when the result is favourable.

Real-world — Fomo3D. A lottery whose airdrop used a predictable, block-derived seed; attackers triggered it only on winning blocks.

3 • Randomness — prevention

Verifiable Random Functions (VRF)

A provider (e.g. Chainlink) returns a random value WITH a cryptographic proof anyone can verify. The de-facto decentralized standard for secure randomness.

PREVRANDAO (PoS)

Derived from the validators' RANDAO process. Manipulating it requires compromising many validators. Commit to a FUTURE block's value, since the current one is already known.

Caveat: RANDAO can be biased by an attacker controlling the last proposers of an epoch. For high-value contracts, weigh the cost of manipulation and prefer a decentralized oracle (VRF).

4 • Unchecked CALL return values

call and **send** do NOT revert on failure — they return **false**. If you forget to check the result, the contract proceeds as if the transfer succeeded.

```
function sendToWinner() public {  
    require(!payedOut);  
    payable(winner).send(winAmount); // return value IGNORED  
    payedOut = true;                 // set even if the send failed  
}
```

If the winner's transfer fails, payedOut still becomes true — and anyone can then claim the “leftover” via `withdrawLeftOver()`. (Cf. Etherpot, King of the Ether.)

4 • Prevention & the ERC-20 caveat

Always check returns

Check the boolean of send / low-level call, no exceptions. Static analysers flag this automatically.

Prefer call + explicit handling

Over send/transfer (2,300-gas) — gives control of gas forwarding and stays compatible with contract / EIP-7702 recipients.

ERC-20 non-conformance. Checking the return is not enough: some tokens return false, some revert, and some (e.g. USDT) return NOTHING. Mixing these with standard calls gives ambiguous behaviour.

Fix: use OpenZeppelin's SafeERC20 (safeTransfer / safeTransferFrom / safeApprove) — it reverts on false and treats “no return value” as success.

5 • Race conditions & front-running

Pending transactions sit in the public mempool before being ordered into a block. Whoever controls ordering (historically via gas price; today via builders/Flashbots bundles) can insert their transaction first.

```
function solve(string memory solution) public {  
    require(hash == keccak256(abi.encodePacked(solution)));  
    payable(msg.sender).transfer(1000 ether); // reward  
}
```

A user broadcasts the correct `solution`; an attacker sees it in the mempool and submits the SAME answer with higher priority. The attacker's tx confirms first and takes the reward; the victim's reverts.

5 • Front-running — prevention

Commit-reveal

Phase 1: submit a hash of your action (hidden). Phase 2: reveal the data. Attackers can't act on what they can't see. Cost: two transactions + latency.

Slippage limits (minAmountOut)

AMM swaps set a minimum acceptable output. Doesn't stop ordering games but caps the profit an attacker can extract.

Sandwich attack (AMM). With a too-low minAmountOut: attacker BUYS before your swap (pushing price up), your swap fills at the inflated price, then they SELL right after — pocketing the spread. Defense: enforce a tight minimum from a trusted/TWAP price.

6 • Denial of Service — unbounded loops

If a function loops over an array that anyone can grow, the gas cost can exceed the block gas limit — making the function permanently uncallable (funds can be trapped).

```
function distribute() public {
    require(msg.sender == owner);
    for (uint i = 0; i < investors.length; i++) // attacker-inflatable array
        transferToken(investors[i], investorTokens[i]);
}
```

Fix — pull over push: don't iterate user-controlled lists. Let each investor call `withdraw()` to claim their own share; paginate long lists. (Even view functions can blow RPC `eth_call` gas caps.)

6 • DoS — other forms

Progress gated on external calls

If state can only advance by sending ether to an address, a recipient that rejects ether (no payable fallback) freezes the contract forever.

Lost / compromised privileged key

If a critical function needs the owner's key and it is lost, those functions become permanently inaccessible (e.g. an ICO that can never be finalized).

Forced / unexpected ether

Ether can be forced in via selfdestruct or pre-deployment sends. Contracts that assume exact internal accounting may misbehave.

Real-world — ZKsync / Gemholic

921 ETH (~\$1.7M) stuck: the contract relied on `transfer()`, which ZKsync Era doesn't support (not fully EVM-equivalent). The team later unlocked it.

Generic defense: enumerate what could go wrong; avoid single points of failure; never assume an external call/recipient cooperates.

PART III

More vulnerability classes

Precision, oracle/price manipulation, input validation, signatures & misconfiguration

7 • Floating point & precision

Solidity has no usable fixed/floating-point types (determinism requires identical results on every node). Developers emulate decimals with integers — and integer division truncates toward zero.

```
uint tokens = msg.value / weiPerEth * tokensPerEth; // divide BEFORE multiply
// 200 wei / 1e18 → 0, then 0 * rate = 0 (deposit lost!)
uint tokens = msg.value * tokensPerEth / weiPerEth; // multiply FIRST → precise
```

Order of operations

Multiply before dividing; choose large numerators (weiPerToken, not tokenPerEth).

High-precision math

Work in extra precision, then scale down. Use Wad (18-dp) / Ray (27-dp): DS-Math, Aave WadRayMath, Solady, OZ Math.

8 • Price / oracle manipulation

DeFi depends on **price oracles**. The attack targets the DATA, not the code: skew the price an asset appears to have, then borrow/trade against the false value. **Flash loans** supply huge, collateral-free capital (repaid in the same tx) to amplify it.

Don't read spot price from an AMM

A pool's instantaneous reserves are trivially moved by one large (flash-loaned) swap. Never feed that into state-changing logic.

Decentralized oracles

Chainlink / RedStone / Pyth: attacker would need to corrupt a majority of nodes.

TWAP oracles

Time-weighted average, excluding the current block → resists flash loans. Tune the window: longer = costlier to manipulate but laggy.

Cross-check & pause

Compare feeds; flag large divergences and pause the protocol.

8 • Price manipulation — Mango Markets

October 2022 — ~\$116M extracted. With ~\$10M across two accounts, the attacker opened a large MNGO perpetual position, then bought MNGO across exchanges to spike the ORACLE price ~2,300%. Using the inflated position as collateral, they borrowed \$116M from the protocol and left.

It wasn't a code “hack”

No bug in the bytecode — the attacker abused legitimate mechanics fed by a manipulable on-chain price.

On-chain negotiation aftermath

The attacker returned part of the funds in a DAO-approved deal; the legal fallout set notable precedents for DAOs.

Note: inferred on-chain prices are dangerous only when used for STATE-CHANGING operations — informational view-only use is not directly exploitable (but beware contracts that consume such getters).

9 • Improper input validation & access control

Never assume inputs are valid — whether from an EOA, another contract, or even your own. Validate ranges, lengths, boundaries and the classic zero-value / zero-address case.

```
// arbitrary external call: attacker supplies the target address
for (uint i; i < claimContracts.length; i++)
    total += IClaim(claimContracts[i]).claimRewards(msg.sender); // untrusted!
IERC20(token).transfer(msg.sender, total); // pays out fabricated rewards
```

Validate & allowlist

Reject the zero address; allowlist trusted targets. Factor checks into modifiers / internal functions.

Access control

msg.sender is just a parameter. Use OZ Ownable / Ownable2Step (safer transfer) or AccessControl (roles).

10 • Signature replay & related pitfalls

Off-chain signatures enable gasless / delegated actions — but a signature, once made, must be usable only once and only where intended. Four distinct pitfalls:

No nonce → replay

Without a per-message nonce, the same signature can be submitted repeatedly to drain a balance.

Signature malleability

Raw ecrecover accepts a tweaked-but-valid variant of a signature. Never use signatures as unique IDs.

abi.encodePacked ambiguity

Packing dynamic arrays can make different inputs hash identically — the signed data can be re-interpreted.

Cross-chain replay

The same signature is valid on every chain unless the signed data binds the chainId (and contract).

10 • Signatures — prevention

Nonces

Include a unique, incrementing nonce in the signed data; the contract records used nonces and rejects reuse.

OZ ECDSA library

Use it instead of raw ecrecover — it rejects malleable (high-s) signatures and handles errors safely.

abi.encode for dynamic data

Unambiguous encoding of arrays/structs — prevents the boundary-collision attack of encodePacked.

EIP-712 typed data

Binds chainId / contract / version (stops cross-chain replay) AND shows the user readable fields — anti-phishing.

Real-world — TCH token (2024). A used-signature mapping was bypassed by malleating v/s: the altered-but-valid signature wasn't recognised as “used”, enabling repeated unauthorized burns.

11 • Misconfiguration

Not a code bug per se — a mistake during deployment, upgrade or governance. Audits often skip deployment/upgrade scripts, so these slip through.

yUSDT (Yearn, 2023)

A storage variable pointed at the **WRONG** underlying token (IUSDC). Unnoticed for 1,000+ days; attacker drained ~\$11.6M.

Ronin Bridge (2024)

An upgrade called `initializeV4` but skipped `initializeV3`, leaving `_totalOperatorWeight` uninitialized. A white-hat MEV bot front-ran and returned 4,000 ETH.

Sonne Finance (2024)

Market-activation steps were queued as **SEPARATE** timelock txs with a public executor role → executed out of order, leaving an empty market → ~\$20M drained.

Prevention

Test & audit deployment/upgrade scripts in a live-like env. Make ordered governance actions **ATOMIC** (e.g. `scheduleBatch()`).

PART IV

Tooling, resources & takeaways

Battle-tested libraries, security wargames, and the big picture

Reuse battle-tested contract libraries

OpenZeppelin

The de-facto standard suite: ERC-20/721/1155, AccessControl/Ownable, Pausable, ReentrancyGuard, SafeERC20, ECDSA, proxies/upgrades, Math. Extensively reviewed.

Solmate (Paradigm)

Lean, opinionated, modern building blocks.

Solady (Vectorized)

Aggressively gas-optimized implementations.

Why reuse?

Code exercised across thousands of deployments is far safer than anything freshly written. Reuse is defensive engineering, not laziness.

Sharpen your skills — security wargames

Ethernaut (OpenZeppelin)

Solidity wargame: hack levels to learn classic vulnerabilities.

Damn Vulnerable DeFi

18 CTF challenges: flash loans, oracles, governance, NFTs.

Capture the Ether

Classic CTF-style Ethereum security puzzles.

QuillCTF

Ethereum security puzzles by QuillAudits.

Paradigm CTF

Annual, advanced; write-ups reflect cutting-edge exploits.

Cyfrin Updraft / Secureum

Structured courses & a bootcamp on auditing.

Key takeaways

- **Assume an adversarial, public world** — Every input is hostile; every function will be probed. Obscurity is not security.
- **Checks-Effects-Interactions + guards** — Update state before external calls; add ReentrancyGuard. This alone kills a whole class of bugs.
- **Don't trust external data** — Oracles can be manipulated (flash loans). Prefer decentralized / TWAP feeds; cross-check and be ready to pause.
- **Reuse audited libraries** — OpenZeppelin et al. over hand-rolled code; SafeERC20, ECDSA, AccessControl, Math, proxies.
- **Mind precision & arithmetic** — Multiply before dividing; high-precision (Wad/Ray); guard vault inflation.
- **Security spans the whole lifecycle** — Most upgrade/deployment incidents are misconfiguration. Test & audit scripts; make ordered actions atomic.

PART V

References

Primary source, standards, libraries and post-mortems

References (1/2)

- [1] Antonopoulos A., Wood G. et al. — Mastering Ethereum, 2nd ed., Ch. 9 “Smart Contract Security” (CC BY-NC-ND). PRIMARY SOURCE. masteringethereum.xyz · github.com/ethereumbook/ethereumbook
- [2] OpenZeppelin Contracts — ReentrancyGuard, SafeERC20, ECDSA, AccessControl, Ownable2Step, Math, ERC-4626. docs.openzeppelin.com/contracts
- [3] Solidity documentation — security considerations, DELEGATECALL, storage layout, units. docs.soliditylang.org
- [4] EIP-1153 — Transient storage opcodes (ReentrancyGuardTransient). eips.ethereum.org/EIPS/eip-1153
- [5] EIP-712 — Typed structured data hashing and signing. eips.ethereum.org/EIPS/eip-712
- [6] EIP-7702 — Set EOA account code (impact on transfer/send). eips.ethereum.org/EIPS/eip-7702
- [7] ERC-20 / ERC-721 / ERC-777 / ERC-4626 token standards. eips.ethereum.org
- [8] Chainlink VRF & Price Feeds; Uniswap V2/V3 TWAP oracles. docs.chain.link · docs.uniswap.org

References (2/2) — incidents, tools & credits

- [9] The DAO (2016) post-mortems; Ethereum/ETC hard fork analyses.
- [10] Parity multisig wallet (2nd hack, 2017) — “Parity Multisig Hacked. Again.”
- [11] Mango Markets (2022) oracle-manipulation exploit & legal proceedings.
- [12] Incident write-ups: Libertify (2023), yUSDT/Yearn (2023), Ronin Bridge (2024), Sonne Finance (2024), ZKsync/Gemholic, TCH token (2024).
- [13] Contract libraries: Solmate (Paradigm), Solady (Vectorized).
- [14] Wargames: Ethernaut, Damn Vulnerable DeFi, Capture the Ether, QuillCTF, Paradigm CTF; courses: Cyfrin Updraft, Secureum.
- [15] Static analysis / tooling: Slither, Mythril, Foundry (fuzz & invariant testing).

Figure credit & licence. Figure 9-1 (read-only reentrancy) is reproduced from Mastering Ethereum, 2nd ed. The work is licensed **CC BY-NC-ND**; the figure is reused with attribution for non-commercial educational use. All other text is original course material.

Questions?

“Code is law” — so write it as if billions depend on it, because one day they might. Defense in depth, audited libraries, and relentless skepticism of every input.

Grazie per l'attenzione



L'Europa investe sul Piemonte, il Piemonte investe su di te

